

Assignment 02 Submission

Name: Emirhan Köksal

Student ID: 20210601040

Git clone command (required format):

```
git clone "https://github.com/koksal100/20210601040_EMIRHAN_KOKSAL.git"  
"20210601040_EMIRHAN_KOKSAL"
```

Part A: Essay

Bir Linux Sistem Yöneticisinin Temel Sorumlulukları

Modern BT ekosisteminde Linux; bulut bilişimin, web sunucularının, kurumsal veri tabanlarının ve DevOps süreçlerinin belkemiğini oluşturur. Bu altyapının operasyonel olarak ayakta kalmasını sağlayan en kritik rol ise Linux Sistem Yöneticisidir (SysAdmin). Bir Linux sistem yöneticisi; Linux tabanlı sistemlerin sağlığını, güvenliğini, performansını ve kesintisiz çalışmasını (uptime) sağlamakla yükümlüdür. Donanım kapasitesi ile yazılım operasyonları arasında köprü kuran bu rol, işletim sistemi çekirdeği (kernel), mimarisi ve komut satırı arayüzü (CLI) hakkında derin bir uzmanlık gerektirir. Ölçek ne olursa olsun, bir sistem yöneticisinin temel görevleri dört ana başlıkta toplanır: kullanıcı yönetimi, süreç (process) kontrolü, dosya sistemi yönetimi ve sistem izleme (monitoring).

1. Kullanıcı ve Grup Yönetimi

Linux'ta güvenlik, kimlik ve erişim yönetimiyle başlar. Linux çok kullanıcı (multi-user) bir işletim sistemi olduğu için, sistem yöneticisinin ilk görevi sisteme kimlerin erişebileceğini ve hangi yetkilere sahip olacağını belirlemektir. Bu, yetkisiz erişimleri engeller ve kritik sistem yapılandırmalarını korur.

Hesap Oluşturma ve Silme: Sistem yöneticileri `useradd` ve `usermod` gibi komutlarla yeni kullanıcı tanımlar, home directory ve varsayılan kabukları (shell) belirler. Bir çalışan şirketten ayrıldığında ise `userdel` ile hesap güvenli bir şekilde kapatılır.

Yetki Ayrımı (Privilege Separation): En az yetki ilkesi (principle of least privilege) doğrultusunda, kullanıcılar rollerine göre (geliştirici, test uzmanı vb.) `groupadd` ile gruplandırılır. Herkese tam yetkili "root" erişimi vermek yerine, `/etc/sudoers` dosyası `visudo` komutuyla düzenlenerek yalnızca belirli kullanıcıların sınırlandırılmış idari komutları çalıştırmasına izin verilir.

Kimlik Doğrulama ve Güvenlik: Şifre geçerlilik sürelerinin belirlenmesi, güçlü şifre politikalarının uygulanması ve SSH anahtar tabanlı (SSH key-based) kimlik doğrulamanın yapılandırılması günlük güvenlik rutinlerinin bir parçasıdır.

2. Process Kontrolü ve Yönetimi

Linux, görevleri "süreç" (process) adı verilen bağımsız yürütme hatları olarak işler. Bir sistem yöneticisi; kaynak tahsisini optimize etmek, sistem kararlılığını korumak ve kilitlenen uygulamalara müdahale etmek için bu süreçleri aktif olarak yönetmelidir.

Sistem Aktivitesini Takip Etme: Sistem yöneticileri; CPU ve bellek tüketimini gerçek zamanlı görmek, sistemi yoran veya yanıt vermeyen işlemleri tespit etmek için ps, top ve htop gibi araçları kullanır.

Sinyal Gönderme ve Süreç Sonlandırma: Bir servis yanıt vermeyi kestiğinde, yönetici kill, pkill veya killall komutlarıyla sürece sinyal gönderir. Örneğin; güvenli bir kapanış için SIGTERM (15), süreci anında ve zorla durdurmak için SIGKILL (9) sinyali verilir.

Otomasyon ve Arka Plan İşleri: Arka planda çalışan servislerin (daemons) yönetimi genellikle systemd mimarisi üzerinden systemctl (start, stop, enable, disable) komutlarıyla yapılır. Ayrıca, günlük yedeklemeler veya log rotasyonları gibi tekrarlayan işleri otomatikleştirmek için cron veya systemd zamanlayıcıları kullanılır.

3. Dosya Sistemi ve Depolama Yönetimi

Veri bütünlüğü ve düzenli depolama, kurumsal uygulamaların sürdürülebilirliği için hayati önem taşır. Linux yöneticileri, işletim sistemine bağlı depolama birimlerini yapılandırmak ve bakımını yapmaktan sorumludur.

Bölümleme (Partitioning) ve Biçimlendirme: Fiziksel depolama alanlarını mantıksal bölümlere ayırmak için fdisk, gdisk veya parted araçları; bu bölümlere dosya sistemi (Ext4, XFS vb.) uygulamak için ise mkfs komutu kullanılır.

Mantıksal Hacim Yönetimi (LVM): Modern sistemlerde yoğun olarak kullanılan LVM, yöneticilerin sistemi kapatmaya gerek duymadan (online) disk alanlarını dinamik olarak yeniden boyutlandırmasına olanak tanır.

Bağlama (Mounting) İşlemleri: Depolama alanlarının kullanılabilmesi için dizin ağacına eklenmesi gerekir. Yöneticiler, /etc/fstab dosyasını yapılandırarak yerel disklerin veya ağ paylaşımlarının (NFS) sistem açılışında otomatik olarak bağlanmasını sağlar.

İzinler ve Sahiplik: Hassas dosyaların güvenliği için chmod ile okuma/yazma/çalıştırma izinleri, chown ile de kullanıcı/grup sahiplikleri titizlikle yönetilir.

4. Monitoring ve Performans Optimizasyonu

Proaktif bir sistem yöneticisi, sistemin çökmesini beklemmez; olası sorunları kullanıcılar etkilenmeden önce yakalamak için sistemi sürekli izler. Sistem izleme; donanım metriklerini, ağ performansını ve yazılım loglarını takip etmeyi kapsar.

Kaynak Metrikleri: Disk G/Ç (I/O) hızları, ağ bandı, bellek ve CPU eşikleri vmstat, iostat ve df (disk free) gibi temel araçlarla izlenir.

Log Analizi: /var/log dizini sistemin günlüğüdür. Sistem yöneticileri donanım arızalarını veya uygulama hatalarını teşhis etmek için grep, tail -f gibi araçlarla veya journalctl (systemd günlükleri için) komutuyla logları analiz eder.

Proaktif Uyarı Sistemleri: Üretim (production) ortamlarında, metrikler güvenli sınırların dışına çıktığında otomatik uyarılar almak için Prometheus, Nagios veya Zabbix gibi merkezi izleme çözümleri entegre edilir.

Linux Sistem Yönetiminde Güvenlik ve Uyumluluk Sorumlulukları

Bilgi güvenliğinin ve yasal uyumluluk standartlarının (KVKK, GDPR, PCI-DSS, ISO 27001 gibi) kritik önem taşıdığı modern kurumsal dünyada, Linux Sistem Yöneticisinin (SysAdmin) rolü sadece sistemlerin ayakta kalmasını sağlamanın çok ötesine geçmiştir. Veri ihlallerinin, fidye yazılımlarının ve siber saldırıların hedefindeki sunucuları korumak, doğrudan sistem yöneticisinin sorumluluğundadır. Bir Linux SysAdmin; kimlik doğrulama mekanizmalarını kurmak, yetkilendirme sınırlarını çizmek ve sistemi olası tehditlere karşı sıkılaştırmak (hardening) gibi proaktif güvenlik adımlarından sorumludur.

1. Kullanıcı Kimlik Doğrulaması (Authentication)

Güvenliğin ilk hattı, sisteme girmeye çalışan kişinin iddia ettiği kişi olduğunu doğrulamaktır. Linux yöneticileri, zayıf kimlik doğrulama yöntemlerinden kaynaklanabilecek siber riskleri minimize etmek için modern protokoller uygular.

SSH Güvenliği ve Anahtar Tabanlı Erişim: Kurumsal ortamlarda şifre ile SSH bağlantısı yapmak büyük bir güvenlik açığıdır. Yöneticiler, şifreli girişleri tamamen kapatarak bunun yerine SSH Anahtar Çiftleri (Public/Private Key) kullanımını zorunlu kılar. Ayrıca SSH portunu değiştirmek ve root kullanıcısının doğrudan SSH üzerinden oturum açmasını engellemek (PermitRootLogin no) standart prosedürdür.

Çok Faktörlü Kimlik Doğrulama (MFA/2FA): Kritik sunuculara erişimde güvenliği artırmak amacıyla, Google Authenticator veya benzeri sistemler üzerinden çalışan PAM (Pluggable Authentication Modules) modülleri entegre edilerek iki aşamalı doğrulama zorunlu hale getirilir.

Merkezi Kimlik Yönetimi: Çok sayıda sunucunun bulunduğu ortamlarda yerel şifre yönetimi sürdürülemez. SysAdmin'ler; LDAP, Active Directory veya FreeIPA gibi merkezi kimlik doğrulama sistemlerini Linux sunuculara entegre ederek tek bir merkezden kullanıcı kontrolü sağlar.

2. Authorization

Kimliği doğrulanan bir kullanıcının veya servisin, sistem içinde tam olarak hangi kaynaklara (dosya, izin, komut) erişebileceğinin belirlenmesi sürecidir.

Gelişmiş Dosya İzinleri ve ACL: Geleneksel Linux okuma/yazma/çalıştırma (rwx) izinleri bazen karmaşık kurumsal ihtiyaçlar için yetersiz kalır. Yöneticiler, ACL (Access Control Lists) kullanarak belirli bir dosyaya veya dizine, geleneksel grup yapılarından bağımsız olarak spesifik kullanıcı bazlı esnek izinler tanımlar (getfacl ve setfacl).

Sudo Politikalarının Yönetimi: Tam yetkili "root" hesabı her zaman koruma altında olmalıdır. Sistem yöneticileri, /etc/sudoers dosyasını yapılandırarak mühendislerin veya uygulamaların sadece işlerini yapmalarına yetecek kadar komutu çalıştırabilmesini sağlar (En Az Yetki İlkesi).

Zorunlu Erişim Kontrolü (MAC - Mandatory Access Control): Geleneksel izinlerin (DAC) bypass edilmesini önlemek amacıyla Linux çekirdeği seviyesinde çalışan SELinux (Security-Enhanced Linux) veya AppArmor gibi güvenlik modülleri aktif tutulur. Bu modüller, bir süreç (process) ele geçirilse bile onun sistemin geri kalanına zarar vermesini engeller.

3. Sistem Sıkılaştırma (System Hardening)

Sistem sıkılaştırma, bir sunucunun saldırı yüzeyini (attack surface) en aza indirmek için yapılan yapılandırma optimizasyonlarının bütünüdür. Ham (default) kurulumla gelen bir Linux dağıtımı, sıkılaştırma yapılmadan üretim ortamına alınamaz.

Gereksiz Servislerin ve Portların Kapatılması: Kullanılmayan her servis ve açık her port, potansiyel bir arka kapıdır. Yöneticiler netstat veya ss komutlarıyla açık portları tarar; ihtiyaç duyulmayan servisleri kalıcı olarak devre dışı bırakır ve kaldırır.

Güvenlik Duvarı (Firewall) Yönetimi: Gelen ve giden ağ trafiğini sıkı bir denetime tabi tutmak için iptables, nftables veya firewalld / ufw gibi araçlar kullanarak katı kurallar tanımlanır. Sadece belirli IP adreslerinin veya alt ağların (subnet) kritik servislere erişimine izin verilir.

Yama Yönetimi (Patch Management): Yazılım açıklarını kapatmanın en etkili yolu sistemleri güncel tutmaktır. SysAdmin'ler güvenlik yamalarını (apt-get upgrade veya dnf update) yakından takip eder ve sistem kesintilerini minimize edecek şekilde otomatik veya planlı yama politikaları geliştirir.

4. Denetim, Log Yönetimi ve Uyumluluk (Compliance)

Sistemlerin güvenli olduğunu iddia etmek yetmez; bunun yasal standartlar çerçevesinde kanıtlanabilmesi ve denetlenebilmesi gerekir.

Sistem Denetimi (Auditing): auditd (Linux Audit Framework) alt yapısı kurularak, kritik sistem dosyalarında (örneğin /etc/passwd veya şifre dosyaları) kimin, ne zaman, hangi değişikliği yaptığı saniye saniye kayıt altına alınır.

Logların Merkezileştirilmesi ve SIEM Entegrasyonu: Sunucu üzerindeki logların (/var/log) kötü niyetli kişiler tarafından silinme riskine karşı, loglar anlık olarak merkezi bir log sunucusuna veya SIEM (Security Information and Event Management) sistemine aktarılır.

Düzenli Güvenlik Taramaları: Sistem yöneticileri, uyumluluk standartlarına (PCI-DSS vb.) bağlı kalabilmek adına OpenVAS, Lynis gibi zafiyet tarama araçlarıyla sistem üzerinde periyodik

kontroller gerçekleştirir ve çıkan raporlara göre sıkılaştırma politikalarını günceller.

Linux Sistem Yönetiminde Sistem Bakımı ve Otomasyon

Büyüyen kurumsal altyapılarda, sistemlerin sadece güvenli olması yetmez; aynı zamanda sürdürülebilir, güncel ve kararlı çalışması gerekir. Linux Sistem Yöneticisinin (SysAdmin) zamanının büyük bir kısmı, sistemlerin sorunsuz ilerlemesini sağlayan rutin bakım işlemlerine ve bu işlemleri insani hatalardan arındırmak için yapılan otomasyon çalışmalarına ayrılır. Manuel olarak yapılan her tekrarlayan iş, zaman kaybı ve hata riskidir. Bu nedenle modern bir Linux yöneticisi; yama yönetimini, yedekleme operasyonlarını ve kabuk programlamayı (shell scripting) iş süreçlerinin merkezine konumlandırır.

1. Yama ve Yazılım Yönetimi (Patch and Software Management)

Yazılım dünyasında açıklar ve hatalar kaçınılmazdır. Sistemlerin kararlılığını korumak ve siber güvenlik risklerini azaltmak için düzenli yazılım güncellemeleri ve yama yönetimi hayati önem taşır.

Paket Yöneticilerinin Kullanımı: Dağıtıma bağlı olarak apt (Debian/Ubuntu) veya dnf / yum (RHEL/CentOS/Fedora) gibi paket yöneticileri kullanılarak sistemdeki tüm yazılımların güncel kalması sağlanır.

Güvenlik Yamaları (Security Patches): Yöneticiler, kritik güvenlik açıklarını (CVE) kapatmak amacıyla sadece güvenlik odaklı yamaları filtreleyerek sisteme geçebilir. Üretim (production) ortamlarında sistem kesintilerini önlemek için bu yamalar önce test ortamlarında denenir.

Canlı Çekirdek Yamalama (Live Kernel Patching): Kesintiye (downtime) tahammülü olmayan kritik sunucularda, sistemi yeniden başlatmadan (reboot) Linux çekirdeğindeki (kernel) güvenlik açıklarını kapatmak için Kpatch veya Canonical Livepatch gibi teknolojiler kullanılır.

2. Yedekleme Operasyonları (Backup Operations)

Veri kaybı, bir şirketin karşılaşılabileceği en büyük felaketlerden biridir. Donanım arızaları, kullanıcı hataları veya siber saldırılara karşı sistem yöneticisi her zaman çalışan bir "B planına" (yedeklemeye) sahip olmalıdır.

Yedekleme Stratejileri: Sektör standardı olan 3-2-1 yedekleme kuralı (verinin 3 kopyası, 2 farklı medya türü, 1 kopya uzak lokasyonda) sıklıkla uygulanır. Tam (Full), Artımlı (Incremental) ve Fark

tabanlı (Differential) yedekleme türleri ihtiyaca göre planlanır.

Temel Yedekleme Araçları: Dosyaları arşivlemek için tar ve gzip/bzip2 kullanılırken, yerel veya uzak sunucular arasında verileri verimli bir şekilde senkronize etmek için yalnızca değişen blokları kopyalayan rsync aracı tercih edilir.

Yedeklerin Doğrulanması: Sadece yedek almak yetmez; sistem yöneticisi bu yedeklerin bozulup bozulmadığını (data corruption) düzenli olarak test etmeli ve geri yükleme (restore) simülasyonları yapmalıdır.

3. Otomasyon Araçları ve Komut Dosyaları (Scripting)

Bir sistem yöneticisinin verimliliği, ne kadar iyi otomasyon yapabildiğiyle doğrudan ilişkilidir.

Tekrarlayan işleri otomatikleştirmek, sistem yöneticisine daha kritik mimari sorunlara odaklanma fırsatı tanır.

Kabuk Programlama (Shell Scripting): Bash veya Sh dilleri kullanılarak yazılan betikler; disk alanı kontrolü, otomatik kullanıcı açma, servislerin durumunu kontrol etme ve raporlama gibi işleri tek tıkla veya tamamen insansız yürütür.

Zamanlanmış Görevler (Cron Jobs): /etc/crontab dosyası veya crontab -e komutu aracılığıyla, hazırlanan script'lerin belirli zaman dilimlerinde (örneğin her gece saat 03:00'te yedek alınması veya her Pazar logların temizlenmesi) otomatik çalışması sağlanır.

Modern Yapılandırma Yönetimi: Çoklu sunucu ortamlarında manuel script yazmak yerine Ansible, Puppet veya Chef gibi altyapıyı kod olarak yöneten (Infrastructure as Code) modern otomasyon araçları kullanılarak yüzlerce sunucu tek bir merkezden dakikalar içinde güncellenebilir.

4. Log Dosyalarının Kritik Önemi

Linux işletim sisteminde ve üzerinde çalışan uygulamalarda gerçekleşen her olay, /var/log dizini altındaki log (günlük) dosyalarına kaydedilir. Loglar, bir sistem yöneticisinin "gözü ve kulağı" konumundadır.

Neden Önemlidir?

Hata Teşhisi (Troubleshooting): Bir servis çöktüğünde veya sistem yavaşladığında, sorunun kök nedenini (root cause) bulmanın ilk ve en kesin yolu journalctl veya /var/log/syslog (messages) dosyalarını incelemektir.

Güvenlik Denetimi: /var/log/auth.log (secure) dosyası, sisteme yapılan başarılı/başarısız tüm giriş denemelerini tutar. Buradaki anormal hareketler, potansiyel bir siber saldırıyı erkenden fark etmeyi sağlar.

Kapasite Planlama: Sistem kaynaklarının (disk, bellek) ne hızla tükendiği log analizleri sayesinde öngörülebilir ve sistem çökmeden donanım takviyesi planlanabilir.

Log Rotasyonu (Logrotate): Log dosyalarının diski tamamen doldurmasını önlemek için logrotate servisi kullanılır. Bu servis, eski logları sıkıştırır, arşivler ve belirli bir süre sonra kalıcı olarak siler.

Part B: Make + ask Pipeline

Part B - Pipeline Çalışma Mantığı (Make + ask)

Bu ödevde GNU Make ile bağımlılık grafiği (DAG) kurularak codebase.txt dosyasından otomatik bir Engineering Action Plan üretilmiştir.

Akış: codebase.txt -> quality.md, perf.md, security.md (paralel) -> quality.sum.md, perf.sum.md, security.sum.md -> concatenated.md -> refined.md -> action.plan.md

Senaryo 1: codebase.txt güncellenirse

- codebase.txt kök girdidir.
- quality.md, perf.md, security.md tekrar üretilir (paralel).
- quality.sum.md, perf.sum.md, security.sum.md tekrar üretilir.
- concatenated.md, refined.md, action.plan.md tekrar üretilir.
- Sonuç: Plan yeni kod tabanına göre baştan oluşur.

Senaryo 2: security.sum.md güncellenirse

- concatenated.md tekrar oluşur.
- refined.md tekrar üretilir.
- action.plan.md tekrar üretilir.
- quality/performance tarafındaki hedefler yeniden çalışmaz.

Senaryo 3: refined.md güncellenirse

- Sadece action.plan.md yeniden üretilir.
- Üst akış dosyaları yeniden çalışmaz.

Neden make -j?

- Bağımsız hedefler aynı anda çalışır, süre kısalır.
- Bağımlı adımlar DAG kurallarına göre doğru sırada bekletilir.

Makefile Content

```
ASK ?= ./ask
CODEBASE ?= codebase.txt

.PHONY: all clean

all: action.plan.md

# Phase 1 - FAN-OUT (parallel)
quality.md: $(CODEBASE)
    @cat $(CODEBASE) | $(ASK) "You are a senior code reviewer. Analyze ONLY code quality: readability, structure, naming, modularity, duplication, maintainability. Output exactly 5-7 bullet points as 'problem -> fix'. Keep each bullet actionable and concise." > $@

perf.md: $(CODEBASE)
    @cat $(CODEBASE) | $(ASK) "You are a performance engineer. Analyze ONLY performance: bottlenecks, algorithmic inefficiencies, memory/I/O hotspots, unnecessary work. Output exactly 5-7 bullet points as 'issue -> optimization'. Keep each bullet actionable and concise." > $@

security.md: $(CODEBASE)
    @cat $(CODEBASE) | $(ASK) "You are an application security reviewer. Analyze ONLY security: vulnerabilities, unsafe patterns, missing validation, secret handling, auth/authz issues. Output exactly 5-7 bullet points as 'risk -> mitigation'. Keep each bullet actionable and concise." > $@

# Phase 2 - LOCAL SUMMARIZATION
quality.sum.md: quality.md
    @cat quality.md | $(ASK) "Summarize into EXACTLY 5 bullets. Keep only actionable code quality items. Format as '- ...' and keep each bullet short." > $@

perf.sum.md: perf.md
    @cat perf.md | $(ASK) "Summarize into EXACTLY 5 bullets. Keep only actionable performance items. Format as '- ...' and keep each bullet short." > $@

security.sum.md: security.md
    @cat security.md | $(ASK) "Summarize into EXACTLY 5 bullets. Keep only actionable security items. Format as '- ...' and keep each bullet short." > $@

# Phase 3 - CONCAT REPORT (no LLM)
concatenated.md: quality.sum.md perf.sum.md security.sum.md
    @{\
        echo "## Code Quality"; \
        cat quality.sum.md; \
        echo ""; \
        echo "## Performance"; \
        cat perf.sum.md; \
        echo ""; \
        echo "## Security"; \
        cat security.sum.md; \
    } > $@

# Phase 4 - FAN-IN #1 (LLM Refine)
refined.md: concatenated.md
```

```
@cat concatenated.md | $(ASK) "Refine this report by removing duplicates, merging overlaps, and keeping only high-signal actionable issues. Preserve these sections: Code Quality, Performance, Security. Keep concise bullet points." > $@
```

```
# Phase 5 - FAN-IN #2 (Final Engineering Action Plan)
```

```
action.plan.md: refined.md
```

```
@cat refined.md | $(ASK) "Create a final 'Engineering Action Plan' in markdown. Include prioritized actions (High/Medium/Low), effort estimate for each action (Small/Medium/Large), and recommended execution order. Keep the plan concrete and implementation-oriented." > $@
```

```
clean:
```

```
@rm -f quality.md perf.md security.md quality.sum.md perf.sum.md security.sum.md concatenated.md refined.md action.plan.md
```